

Part C – Configuration update warnings (REQ 10)

Six steps. One new class, three warning labels, the confirmation and the acknowledge, and the property keys. Every file it touches is one you have already sent, so each step names the class and the exact line to add after.

The decision, taken from the requirement

The requirement says the deviations are closed when a **new configuration is applied** to the aircraft. So the trigger is the health configuration file, not the aircraft update in general: editing a register number or a description leaves every deviation untouched. In the code that is the test `acConfigDto.getHealthConfigurationFile() != null` on an update, used in Step 6.

Action	Deviations
Update Aircraft, import a health configuration file	All active ones are closed
Update Aircraft, change only the register number or description	Untouched
Create a new aircraft	Nothing to close
Relearn a threshold	Untouched – Step 5 says so on screen

The six steps

Step 1 — DeviationWarningUtils – new class

`mgs-client/.../presentation/utils/DeviationWarningUtils.java`

Create it beside FlightUtils, in the same package. Two helpers used by the steps below.

```

package com.eurocopter.mgs.presentation.utils;

import java.util.ArrayList;
import java.util.List;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.osgi.framework.BundleContext;
import org.osgi.framework.FrameworkUtil;

import com.eurocopter.mgs.business.health.api.IHealthCommonBusiness;
import com.eurocopter.mgs.service.health.common.dto.IndicatorDeviationDto;

/**
 * BCR-6605 helpers for the configuration update warnings.
 */
public final class DeviationWarningUtils {

    private static final Logger LOGGER = LogManager.getLogger(DeviationWarningUtils.class);

    private DeviationWarningUtils() {
    }

    private static IHealthCommonBusiness business() {
        BundleContext context = FrameworkUtil.getBundle(IHealthCommonBusiness.class).getBundleContext();
        return context.getService(context.getServiceReference(IHealthCommonBusiness.class));
    }

    /** True when the aircraft has at least one deviation that has not been closed. */
    public static boolean hasActiveDeviations(String aircraftId) {
        try {
            return !business().findActiveDeviations(aircraftId).isEmpty();
        } catch (Exception e) {
            LOGGER.warn("Could not check active deviations", e);
            return false;
        }
    }

    /** Closes every active deviation of the aircraft, after a new configuration is applied. */
    public static void acknowledgeAll(String aircraftId) {
        try {
            List<String> ids = new ArrayList<>();
            for (IndicatorDeviationDto dto : business().findActiveDeviations(aircraftId)) {
                ids.add(dto.getId());
            }
            if (!ids.isEmpty()) {
                business().acknowledgeDeviations(ids);
            }
        } catch (Exception e) {
            LOGGER.error("Could not acknowledge deviations after configuration update", e);
        }
    }
}

```

If `IHealthCommonBusiness` does not resolve, that bundle does not import the package yet. Open its `MANIFEST.MF`, `Dependencies` tab, and add `com.eurocopter.mgs.business.health.api` to `Imported Packages`.

Step 2 — AircraftDetailsComposite – the Update Aircraft warning

`mgs-client/.../presentation/aircraft/details/AircraftDetailsComposite.java`

In the **constructor**, add after the call to `initDetailsField(composite, aircraftTypes);` and before `// Dispose resources.`

```

// BCR-6605 warn that applying a new configuration will close the deviations
if (update && DeviationWarningUtils.hasActiveDeviations(aircraftModel.getAircraft().getId())) {
    Label deviationWarning = new Label(this, SWT.WRAP);
    deviationWarning.setText(Messages.CORE.getString("deviationConfigUpdateWarning"));
    deviationWarning.setForeground(getDisplay().getSystemColor(SWT.COLOR_DARK_RED));
    GridData warningData = new GridData(SWT.FILL, SWT.BEGINNING, true, false);
    warningData.widthHint = 440;
    deviationWarning.setLayoutData(warningData);
}

```

Import: `com.eurocopter.mgs.presentation.utils.DeviationWarningUtils`. `Label`, `SWT`, `GridData` and `Messages` are already imported in this class.

Step 3 — ImportConfigurationComposite – the Import configurations warning

mgs-client/.../presentation/aircraft/management/composite/ImportConfigurationComposite.java

In **initAircraftConfigurations**, add immediately after

`top.setLayout(GridLayoutFactory.fillDefaults().numColumns(1).create());` so the warning sits above the file sections.

```
// BCR-6605 warn that importing a new configuration will close the deviations
if (aircraftDetailsComposite.isUpdate() && DeviationWarningUtils.hasActiveDeviations(
    aircraftDetailsComposite.getAircraftModel().getAircraft().getId())) {
    org.eclipse.swt.widgets.Label deviationWarning = pFormToolkit.createLabel(
        top, Messages.CORE.getString("deviationConfigImportWarning"), SWT.WRAP);
    deviationWarning.setForeground(Display.getCurrent().getSystemColor(SWT.COLOR_DARK_RED));
    deviationWarning.setLayoutData(
        GridDataFactory.fillDefaults().grab(true, false).hint(520, SWT.DEFAULT).create());
}
```

Import: `com.eurocopter.mgs.presentation.utils.DeviationWarningUtils`. The `Label` type is written out in full because this class does not import it; add the import instead if you prefer.

Step 4 — LearningPastHoursSelectionComposite – the relearn note

mgs-health-app/.../health/presentation/composite/LearningPastHoursSelectionComposite.java

In the **constructor**, add immediately before `this.pack();` on the last line.

```
// BCR-6605 relearning does not close a deviation
Label deviationNote = new Label(this, SWT.WRAP);
deviationNote.setText(Messages.CORE.getString("deviationRelearnNote"));
deviationNote.setLayoutData(new GridData(SWT.FILL, SWT.BEGINNING, true, false));
```

No imports needed. This note always shows: it is reassurance rather than a warning, and the composite has no aircraft to check against.

Step 5 — OkCreateUpdateListener – the confirmation

mgs-client/.../presentation/aircraft/details/OkCreateUpdateListener.java

In **widgetSelected**, add after the `acConfigDto` block and before the comment `// Validate configuration files`. The context lines are shown so the position is unambiguous.

```
// Get ConfigurationDto parameter
ConfigurationDto acConfigDto = acDetailsComposite.getAircraftConfigurationModel();
if (acConfigDto == null) {
    acConfigDto = new ConfigurationDto();
}

// ===== BCR-6605 add from here =====
// A new health configuration closes every active deviation, so ask first.
// Only a health configuration file triggers this: changing the register number
// or the description leaves the deviations alone.
boolean healthConfigurationImported = AircraftDetailsDialog.isUpdate()
    && aircraftTechnicalId != null
    && acConfigDto.getHealthConfigurationFile() != null;

if (healthConfigurationImported
    && DeviationWarningUtils.hasActiveDeviations(aircraftTechnicalId)
    && !MessageDialog.openConfirm(aircraftDetailsDialog.getShell(),
        Messages.CORE.getString("deviationConfigUpdateConfirmTitle"),
        Messages.CORE.getString("deviationConfigUpdateConfirm"))) {
    return;
}
// ===== to here =====

// Validate configuration files
```

Imports: `org.eclipse.jface.dialogs.MessageDialog` and `com.eurocopter.mgs.presentation.utils.DeviationWarningUtils`.

return leaves the dialog open with nothing saved, so the user can clear the configuration file and press OK again.

5.2 The acknowledge, in the same method. Add between the end of the if / else that performs the update and `aircraftDetailsDialog.close();`.

```
aircraftManagementResult = aircraftManagementBusiness.updateAircraft(
    acDto, acConfigDto, rtbDto, applicableFlightId);
} else {
    aircraftManagementResult = aircraftManagementBusiness.createAircraft(acDto, acConfigDto, rtbDto);
}

// ===== BCR-6605 add from here =====
// The configuration has been replaced, so close the deviations it invalidates.
if (healthConfigurationImported) {
    DeviationWarningUtils.acknowledgeAll(aircraftTechnicalId);
}
// ===== to here =====

aircraftDetailsDialog.close();
```

It sits inside the try block on purpose: if the update throws, the catch runs instead and the deviations are left alone. Closing them after a failed update would be the worst outcome, since the configuration would still be the old one.

Step 6 — CoreMessages.properties – and every language file

mgs-client-common/src/com/eurocopter/mgs/core/presentation/i18n/CoreMessages.properties

Five keys. Place them near the other deviation keys.

```
deviationConfigUpdateWarning=This aircraft has active deviations. Applying a new configuration \\  
will close them, and the affected indicators will return to their configuration values.  
deviationConfigImportWarning=This aircraft has active deviations. Importing a new configuration \\  
will close them, and the affected indicators will return to their configuration values.  
deviationRelearnNote=Relearning a threshold does not close any deviation set on it. The \\  
approved deviation is preserved.  
deviationConfigUpdateConfirmTitle=Active deviations  
deviationConfigUpdateConfirm=This aircraft has active deviations. Applying a new configuration \\  
will close them. Do you want to continue?
```

The trailing backslashes continue a line in a properties file. Write each on one line instead if you prefer.

Build and test

#	Action
1	Rebuild the client only. No server or database change in this set.
2	Create a deviation on an aircraft, then open A/C management and update that aircraft – the red warning shows under the fields
3	Press Import configurations – the same warning shows above the file sections
4	Change only the description and press OK – no confirmation, and <code>SELECT * FROM public.indicator_deviation</code> still shows <code>end_date</code> null
5	Update again, this time importing a health configuration file, and press OK – the confirmation appears
6	Press No – the dialog stays open and nothing is saved
7	Press OK then Yes – the update runs and <code>end_date</code> is now set on every deviation of that aircraft
8	Open Health for that aircraft – the rows are no longer green, they show the new configuration values, and the deviations appear in Deviation history
9	Open the threshold learning dialog – the relearn note shows at the bottom

Item 4 is the one that proves the decision at the top of this document: a routine edit must not close an approved deviation.